

mawk — a feature-rich AWK interpreter in portable C

A single-file, dependency-free implementation of the AWK programming language, written in portable C (compiles cleanly under both MSVC and gcc/clang with `-Wall -Wextra`, no warnings). It covers the vast majority of POSIX AWK, including user-defined functions, `getline`, I/O redirection, a real regex engine with alternation and grouping, and full `RS`/`FS`/`ARGV` handling.

Contents

- [Features](#)
- [Build](#)
- [Usage](#)
- [Language reference](#)
- [Examples](#)
- [Known limitations](#)
- [Architecture notes](#)

Features

- **Patterns & rules** — `BEGIN`, `END`, bare `{ action }` blocks, expression patterns, `/regex/` patterns, and range patterns (`/start/,/end/`).
- **Fields** — `$0`, `$1..$NF`, reading and assigning to any field or to `$0` or `NF` directly, with automatic `OFS`-based rebuild of `$0` on field mutation.
- **Built-in variables** — `FS` `OFS` `ORS` `RS` `SUBSEP` `CONVFMT` `OFMT` `NR` `FNR` `NF` `FILENAME` `RSTART` `RLENGTH` `ARGC` `ARGV` `ENVIRON`.
- **Expressions** — assignment (`=` `+=` `-=` `*=` `/=` `%=` `^=`), the `?:` ternary operator, `||` `&&` `!`, relational and match operators (`<` `<=` `>` `>=` `==` `!=` `~` `!~`), string concatenation, arithmetic (`+` `-` `*` `/` `%` `^`), unary `+`/`-`, pre/post `++`/`--`, and array membership tests (`(key in arr)`, `(a,b) in arr` for multi-subscript arrays).
- **Statements** — `if`/`else`, `while`, `do`/`while`, `for(;;)`, `for (k in arr)`, `break`, `continue`, `next`, `nextfile`, `exit`, `delete arr`/`delete arr[k]`, `return`.
- **Output** — `print` and `printf`, with `>`, `>>`, and `|` redirection to files or commands.
- **getline** — all four POSIX forms: plain `getline`, `getline var`, `getline [var] < file`, and `cmd | getline [var]`, each updating `NR`/`FNR` per the POSIX table.
- **User-defined functions** — `function name(params) { ... }`, recursion, arrays passed by reference, scalars passed by value, and unused extra parameters usable as local scratch scalars/arrays.
- **Built-in functions** — `length` `substr` `index` `split` `sub` `gsub` `match` `sprintf` `toupper` `tolower` `sin` `cos` `atan2` `sqrt` `exp` `log` `int` `rand` `srand` `system` `close` `fflush`.

- **Regex engine** — a real recursive-descent-parsed, backtracking matcher supporting literals, `(.)`, `(^)`, `($)`, `(*)`, `(+)`, `(?)`, `[...]`/`[^...]` classes, grouping `(...)`, and alternation `a|b`. Used everywhere a pattern is needed: `/ere/` rules, `~`/`!~`, `FS`, `RS`, `split()`, `sub()`/`gsub()`, `match()`.
- **Command line** — `-v var=value`, `-F fs`, and repeatable `-f progfile` (both attached `-Fx`/`-vX=Y` and separated `-F x`/`-v X=Y` forms work), `var=value` assignments interleaved with filenames, and full `ARGC`/`ARGV` mutation support from `BEGIN`.
- Tolerant of both Unix (LF) and Windows (CRLF) line endings in the AWK program source itself, so `.awk` files edited on Windows work unmodified.

Build

MSVC (recommended flags — compiles the file as C++ for stricter checking):

```
cl /TP /EHsc /O2 mawk.c /Fe:mawk.exe
```

or, using MSVC's C11 mode:

```
cl /std:c11 /O2 mawk.c /Fe:mawk.exe
```

gcc / clang:

```
gcc -std=c11 -D_POSIX_C_SOURCE=200809L -O2 -o mawk mawk.c -lm
```

No external dependencies — only the C standard library and libm.

Usage

```
mawk [-F fs] [-v var=val ...] 'program text' [file|var=val ...]
mawk [-F fs] [-v var=val ...] -f prog.awk [-f prog2.awk ...] [file|var=val ...]
```

Option	Meaning
<code>-F fs</code>	Set the field separator (<code>FS</code>). Supports <code>\t</code> / <code>\n</code> escapes.
<code>-v var=val</code>	Assign <code>var</code> before <code>BEGIN</code> runs. Repeatable.
<code>-f progfile</code>	Read the program from a file instead of the command line. Repeatable — files are concatenated.
<code>--</code>	End of options.
<code>file</code>	Input file. <code>-</code> means stdin. Multiple files are processed in order, updating <code>FILENAME</code> / <code>FNR</code> .
<code>var=val</code>	Interleaved with files, assigns <code>var</code> at the point it's reached in <code>ARGV</code> processing (matches real awk).

If no files are given (or an `awk` script explicitly reads from `ARGV` without touching it, per convention), input is read from stdin.

Language reference

Default variable values

Variable	Default
<code>FS</code>	<code>" "</code> (whitespace-run splitting)
<code>OFS</code>	<code>" "</code>
<code>ORS</code>	<code>"\n"</code>
<code>RS</code>	<code>"\n"</code>
<code>SUBSEP</code>	<code>"\x1c"</code>
<code>CONVFMT</code>	<code>"%.6g"</code>
<code>OFMT</code>	<code>"%.6g"</code>

`RS` modes

- `RS == "\n"` (default) — one record per line, streamed efficiently.
- Single character other than `\n` — records split on that character.

- `RS == ""` — **paragraph mode**: records are blank-line-separated paragraphs; leading blank lines are skipped.
- Multi-character `RS` — treated as a **regular expression** delimiter, matched incrementally against a growing read buffer.

`getline` forms and what they update

Form	Sets
<code>getline</code>	<code>\$0</code> , <code>NF</code> , <code>NR</code> , <code>FNR</code>
<code>getline var</code>	<code>var</code> , <code>NR</code> , <code>FNR</code>
<code>getline < file</code>	<code>\$0</code> , <code>NF</code>
<code>getline var < file</code>	<code>var</code>
<code>cmd getline</code>	<code>\$0</code> , <code>NF</code> , <code>NR</code>
<code>cmd getline var</code>	<code>var</code> , <code>NR</code>

User-defined functions

```
awk
function max(a, b) {
    return a > b ? a : b
}

function fill_squares(arr, i) {    # 'i' declared as an unused extra
    for (i = 1; i <= 10; i++)      # parameter -> usable as a local var
        arr[i] = i * i
}
```

Arrays are passed by reference (mutations are visible to the caller); scalars are passed by value. Functions may recurse.

Examples

Print the second field of every line:

```
mawk '{ print $2 }' data.txt
```

Sum a column and print the total at the end:

```
mawk '{ sum += $2 } END { print "total:", sum }' data.txt
```

Field-separated on `(:)`, uppercase the first field:

```
mawk -F: '{ print toupper($1) }' /etc/passwd
```

Count word frequency:

```
mawk '{ for (i = 1; i <= NF; i++) count[$i]++ }  
      END { for (w in count) print w, count[w] }' text.txt
```

Read from a file with `(getline)` inside `(BEGIN)`:

```
mawk 'BEGIN { while ((getline line < "list.txt") > 0) print "got:", line  
                close("list.txt") }'
```

Pipe output through an external sort:

```
mawk '{ print $1 | "sort" }' data.txt
```

A small recursive function:

```
mawk 'function fact(n) { return n <= 1 ? 1 : n * fact(n - 1) }  
      BEGIN { print fact(10) }'
```

Known limitations

Explicitly out of scope for this build (documented in the source header):

- True POSIX locale collation in bracket expressions (`([...])` is byte-range only, no locale awareness).
- Regex backreferences.
- Coprocesses (`(|&)`).
- `(printf)`'s dynamic width/precision (`(%*d)`).
- POSIX character classes inside brackets (e.g. `([:alpha:])`).
- Anchors (`(^)`/`($)`) are only honored at the top level of a pattern, not inside a nested group.

Additionally, for very long field or variable values, a handful of internal code paths use fixed-size stack buffers (typically 512 B-4 KB) with `(snprintf)`-based truncation rather than

dynamic growth — this avoids any buffer overflow, but extremely long values in those specific paths (e.g. the subject string inside `sub()`/`gsub()`) may be truncated rather than processed in full. `MAXFIELDS` is capped at 8192.

Architecture notes

The interpreter is a straightforward three-stage pipeline in one file:

1. **Lexer** — hand-written, single-token lookahead, with `/` disambiguated between "start of regex" and "division" based on whether the previous token could end a value expression.
2. **Parser** — recursive descent, building an expression AST (`Node`) and a statement tree (`Stmt`), plus flat lists of pattern/action `Rule`s and `FuncDef`s.
3. **Evaluator** — a tree-walking `eval()`/`exec_stmt()` pair with no intermediate bytecode.

Variables and array contents are stored as linked lists (not hash tables), so lookup is $O(n)$ in the number of live variables/entries — simple and correct, with a cost that only becomes noticeable with very large symbol tables or array sizes. The regex engine is a continuation-passing backtracking matcher over a parsed alternation/concatenation/atom tree (rather than a compiled DFA), which is what allows grouping and alternation to compose cleanly with quantifiers.